



Linear Combination of Non-Unitaries Decomposition

Abeynaya Gnanasekaran · IonQ

Amit Surana · RTX Technology Research Center

Reuben Demirdjian · U.S. Naval Research Laboratory

Thomas Hogancamp · U.S. Naval Research Laboratory

IEEE QCE 2026

Tutorial Objectives and Expected Outcomes

Objective. Introduce the linear combination of non-unitaries (LCNU) decomposition: a more efficient alternative to LCU for quantum linear algebra.

- The theory, and the methods that compute a decomposition.
- Use in hybrid and in fault-tolerant algorithms, differential equations included.
- Hands-on code.

What you take away.

- Where LCNU beats the alternatives, and where it does not apply.
- A new decomposition strategy for your toolkit.
- An active area, with open problems and real applications.

Tutorial Agenda: Session 1 (10:30am-12:00pm)

Focus on background, theory, circuit construction, and basic applications of LCNU decomposition, and hands-on coding exercise:

- Basics of standard Pauli basis based LCU and motivation for using Sigma basis based LCNU (10 min).
- Mathematical properties of Sigma basis and methods for computing LCNU decomposition (10 min).
- Algorithm for efficient quantum circuit construction implementing LCNU (15 min).
- Utilization of LCNU framework within hybrid quantum-classical and fully fault-tolerant algorithms for linear algebra (20 min).
- Applications to linear ODEs/PDEs including an end-to-end resource estimation (20 min).
- Pros/cons of LCNU compared with alternative methods
- Guidelines on how to identify problems where the LCNU technique could be beneficial (5 min).

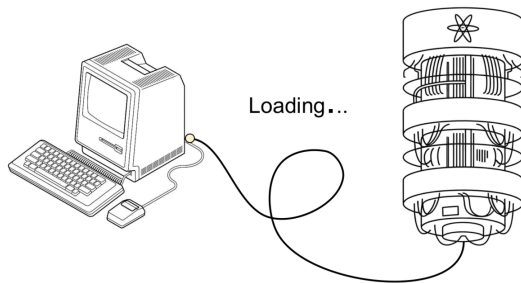
Tutorial Agenda: Session 2 (1:30pm-3:00pm)

Focus on generalization of the Sigma basis based LCNU concept, applications to nonlinear PDEs and fluid problems, and hands-on coding exercise:

- Quick recap of session I (5 min).
- Framework to generalize Sigma basis based LCNU by incorporating other unitary and non-unitary operators to expand/modify the Sigma basis and tailor the decomposition for specific problems (10 min).
- Application to nonlinear PDEs including introduction to Carleman linearization technique (15 min).
- End-to-end case study and hands-on coding exercise for simulating fluid flows (e.g. Burgers equation, Lattice Boltzman Method) with explicit circuit construction, resource estimates, and executing prototypical examples on quantum simulators (50 min).
- Conclusions and avenues for future work (10 min).

Typical Quantum Computing Workflow

- All these steps must be efficient for quantum advantage
 - Load classical data
 - Execute quantum algorithm (circuit)
 - Measure outcomes



Data Loading for Linear Algebra on Quantum Computers

- Linear algebra problems are ubiquitous,

$$\mathbf{Ax} = \mathbf{b}.$$

- Quantum linear system algorithms (HHL and variants) promise computational advantage
 - $\mathbf{A}$ is generally non-unitary, and appropriate data access/loading methods are needed, e.g.
 - Oracles
 - Block encoding, e.g., via Linear combination of unitary (LCU) decomposition
 - Quantum data access models need to be efficient to maintain the advantage
- In many applications, e.g., solutions of partial differential equations (PDEs) and ordinary differential equations (ODEs), $\mathbf{A}$ is typically sparse/structured, e.g., diagonal, block diagonal, banded, Toeplitz:

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & & 0 \\ -1 & \ddots & & \\ & & \ddots & -1 \\ 0 & & -1 & 2 \end{bmatrix}.$$

- Can the structure/sparsity be leveraged to gain efficiency in LCU-based data loading?

LCU Decomposition: A Versatile Data Access Model

- Given a matrix $\mathbf{A} \in \mathbb{C}^{N \times N}$, where $N = 2^n$, its linear combination of unitary (LCU) decomposition is defined as

$$\mathbf{A} = \sum_{l=1}^{n_l} \alpha_l \mathbf{U}_l,$$

where $\alpha_l \in \mathbb{C}$, $\mathbf{U}_l \in \mathbb{C}^{2^n \times 2^n}$ are unitary matrices (i.e. $\mathbf{U}_l \mathbf{U}_l^* = \mathbf{U}_l^* \mathbf{U}_l = \mathbf{I}$) and n_l is # LCU terms.

- Most common approach for LCU decomposition uses Pauli basis $\mathbb{P} = \{\mathbf{I}, \sigma_x, \sigma_y, \sigma_z\}$ (or $\mathbf{I}, X, Y, Z$)

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \quad \sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \mathbf{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$

so that

$$\mathbf{U}_l = \sigma_{n-1} \otimes \sigma_{n-2} \otimes \cdots \otimes \sigma_0$$

with $\sigma_i \in \mathbb{P}$, $i = 0, \dots, n-1$.

- LCU decomposition is a commonly used “data access model” in
 - Variational quantum algorithms (VQA): VQE, VQLS, etc.
 - Fault tolerant algorithms: block encoding based QSP, QVST, etc.

LCU Decomposition in Variational Quantum Algorithms

- Consider the problem of computing ground state of a given Hamiltonian $\mathbf{H}$ (Hermitian, i.e. $\mathbf{H}^* = \mathbf{H}$)

$$\mathbf{H} |\psi_0\rangle = E_0 |\psi_0\rangle$$

- Variational quantum eigensolver (VQE) uses an ansatz state $|\psi(\theta)\rangle$, and classically optimizes the parameters θ

$$\min_{\theta} E(\theta) \equiv \langle \psi(\theta) | \mathbf{H} | \psi(\theta) \rangle \geq E_0$$

- To compute $E(\theta)$, LCU decomposition of $\mathbf{H} = \sum_{i=1}^{n_l} \alpha_i \mathbf{U}_i$ can be used

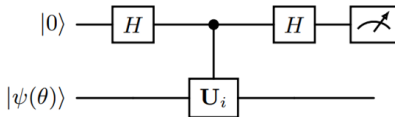
$$\langle \psi(\theta) | \mathbf{H} | \psi(\theta) \rangle = \sum_{i=1}^{n_l} \alpha_i \langle \psi(\theta) | \mathbf{U}_i | \psi(\theta) \rangle$$

where each term $\langle \psi(\theta) | \mathbf{U}_i | \psi(\theta) \rangle$ can be estimated via the Hadamard test.

- Larger $n_l \Rightarrow$ larger # of Hadamard tests needed



$$\mathcal{H}_{\text{el}} |\Psi_n\rangle = E_n |\Psi_n\rangle$$
$$\mathcal{H}_{\text{el}} = - \sum_i \frac{\nabla_{\mathbf{r}_i}^2}{m_e} - \sum_I \sum_i \frac{Z_I e^2}{|\mathbf{R}_I - \mathbf{r}_i|} + \sum_i \sum_{j>i} \frac{e^2}{|\mathbf{r}_i - \mathbf{r}_j|}.$$



LCU Decomposition in Fault Tolerant Algorithms

- Block encoding of a matrix $\mathbf{A}$ involves

$$\mathbf{U} = \begin{bmatrix} \mathbf{A} & \star \\ \star & \star \end{bmatrix},$$

whose application on an ancilla system of the form $|0\rangle |\psi\rangle$ gives desired action

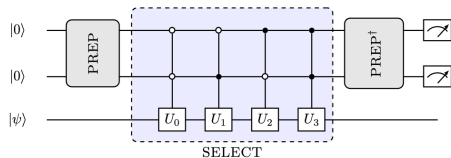
$$\mathbf{U} |0\rangle |\psi\rangle = |0\rangle \mathbf{A} |\psi\rangle + |1\rangle \star |\psi\rangle.$$

- LCU based block encoding can be accomplished by using the PREP and SEL oracles

$$\text{PREP} |0\rangle = \sum_{k=1}^{n_l} \sqrt{\frac{|\alpha_k|}{\lambda}} |k\rangle, \quad \text{SEL} |k\rangle |\psi\rangle = |k\rangle \mathbf{U}_k |\psi\rangle,$$

where $\lambda = \sum_{i=1}^{n_l} |\alpha_i|$ leading to $\langle 0 | \text{PREP}^* \cdot \text{SEL} \cdot \text{PREP} |0\rangle |\psi\rangle = \frac{\mathbf{A} |\psi\rangle}{\lambda}.$

- Larger $n_l \Rightarrow$ increased quantum resources (i.e., #ancillas, #gates and circuit depth) needed for the implementation of the associated circuit



Pauli Basis LCU Decomposition

- $U_l = \sigma_{n-1} \otimes \sigma_{n-2} \otimes \cdots \otimes \sigma_0$ with $\sigma_i \in \mathbb{P}$, $i = 0, \dots, n-1$ can be implemented efficiently on a quantum device.
- In applications, such as quantum chemistry, the Pauli decomposition is often known by construction.
- More generally, numerical techniques have to be employed ¹.
- In general the # Pauli LCU terms, n_l , can vary as $O(2^n)$ for a diagonal matrix and $O(2^{2n})$ for a dense one.
- For sparse matrices that typically arise in solving PDEs/ODEs, n_l can be $> \text{nnz}(\mathbf{A})$, where nnz is the number of non-zero terms in $\mathbf{A}$.
- Larger $n_l \Rightarrow$ more quantum resource needed for data loading, which can become a key bottleneck in using LCU and impact the quantum advantage in many applications
- By using a different basis we show that the poly-logarithmic dependence $n_l = O(\text{poly}(\log N)) = O(\text{poly}(n))$ can be achieved for sparse/structured matrices, where $N = 2^n$ is the matrix size.

¹L. Hantzko, L. Binkowski, and S. Gupta, Tensorized pauli decomposition algorithm, arXiv preprint arXiv:2310.13421, 2023.

Sigma Basis and LCNU Decomposition

- Sigma basis^{2 3} is a set including non-unitary operators

$$\mathbb{S} = \{\mathbf{I}, \sigma_+, \sigma_-, \sigma_+\sigma_-, \sigma_-\sigma_+\},$$

where

$$\sigma_+ = |0\rangle\langle 1| = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \sigma_- = |1\rangle\langle 0| = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad \sigma_+\sigma_- = |0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad \sigma_-\sigma_+ = |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}.$$

- Given a matrix $\mathbf{A} \in \mathbb{C}^{2^n \times 2^n}$, its **linear combination of non-unitary (LCNU)** decomposition is defined as

$$\mathbf{A} = \sum_{l=1}^{n_l} \alpha_l \mathbf{A}_l,$$

where $\alpha_l \in \mathbb{C}$ and $\mathbf{A}_l = \sigma_{n-1} \otimes \sigma_{n-2} \otimes \cdots \otimes \sigma_0$ with $\sigma_i \in \mathbb{S}$, $i = 0, \dots, n-1$.

- Sigma basis offers an efficient decomposition of sparse/structured matrices.
- But this reduction in n_l should not come at the expense of proportionate increase in quantum resources for implementing these non-unitary operators.

² Hai-Ling L., et. al. "Variational quantum algorithm for the Poisson equation." Phys. Rev. A 104.2, 2021

³ A. Gnanasekaran and A. Surana. "Efficient quantum access model for sparse structured matrices using linear combination of things", Phys. Rev. A 113.2, 2026.

Sigma Basis Decomposition, Motivating Example

- Consider the decomposition of the following sparse matrix in Pauli and sigma basis, respectively

$$\begin{aligned} \text{Pauli: } \begin{bmatrix} & & 1 \\ & 0 & \\ 0 & & \\ 2 & & \end{bmatrix} &= 0.75 \sigma_x \otimes \sigma_x - 0.25i \sigma_x \otimes \sigma_y - 0.25i \sigma_y \otimes \sigma_x - 0.75 \sigma_y \otimes \sigma_y \\ \text{Sigma: } \begin{bmatrix} & & 1 \\ & 0 & \\ 0 & & \\ 2 & & \end{bmatrix} &= \sigma_+ \otimes \sigma_+ + 2 \sigma_- \otimes \sigma_- . \end{aligned}$$

- # terms Pauli basis decomposition is 4, whereas sigma basis decomposition require only 2 terms.
- In general, $n_l \leq \text{nnz}(A)$ with sigma basis as desired.
- Next we consider examples of linear/nonlinear PDEs in 1D (results generalize to higher dimensions under some assumptions, see the paper for details).

Sigma Basis Decomposition: Poisson (Elliptic) PDE

- Consider 1D Poisson equation:

$$-\frac{\partial^2 u(x)}{\partial x^2} = f(x), \quad x \in \Omega = [0, l]$$

$$(\text{b.c.}): u(0) = 0, \quad u(l) = 0.$$

- Let $\mathbf{A}\mathbf{u} = \mathbf{b}$ be the linear system generated by central difference discretization scheme, where

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & & & 0 \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & \ddots & 2 & -1 \\ 0 & & & -1 & 2 \end{bmatrix} \in \mathbb{R}^{N_x \times N_x}, \quad (1)$$

is a $N_x \times N_x$ matrix with N_x being # of spatial discretization points and $\mathbf{b} = (\Delta x)^2 (f(x_1), \dots, f(x_{N_x}))^T$.

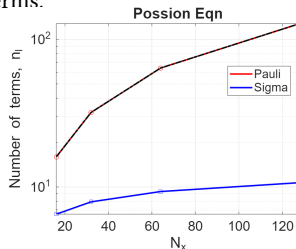
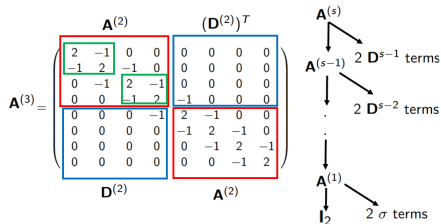
Sigma Basis Decomposition: Poisson PDE

- For simplicity assume $N_x = 2^s$ for some integer $s > 0$.
- We can write a recursive decomposition for $\mathbf{A}$ in sigma basis as follows

$$\mathbf{A} := \mathbf{A}^{(s)} = \begin{bmatrix} \mathbf{A}^{(s-1)} & (\mathbf{D}^{(s-1)})^T \\ \mathbf{D}^{(s-1)} & \mathbf{A}^{(s-1)} \end{bmatrix} = \mathbf{I}_2 \otimes \mathbf{A}^{(s-1)} + \sigma_- \otimes \mathbf{D}^{(s-1)} + \sigma_+ \otimes (\mathbf{D}^{(s-1)})^T,$$

$$\mathbf{D}^{(s-1)} = \begin{bmatrix} 0 & \dots & -1 \\ \vdots & \ddots & \vdots \\ 0 & \dots & 0 \end{bmatrix} = -\underbrace{\sigma_+ \otimes \dots \otimes \sigma_+}_{s-1 \text{ times}}, \quad \mathbf{A}^{(1)} = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} = 2\mathbf{I}_2 - \sigma_- - \sigma_+.$$

- Thus, $\mathbf{A}$ can be decomposed into $2s + 1 = 2 \log N_x + 1$ terms.



Sigma Basis Decomposition: Computational Methods

- Given $\mathbf{A}$, how to determine is LCNU decomposition in sigma basis, i.e.

$$\mathbf{A} = \sum_{l=1}^{n_l} \alpha_l \mathbf{A}_l,$$

where $\mathbf{A}_l = \sigma_{n-1} \otimes \sigma_{n-2} \otimes \cdots \otimes \sigma_0$ with $\sigma_i \in \mathbb{S}$, $i = 1, \dots, n$.

- For a matrix with single non-zero entry, following theorem gives a simple procedure.

Theorem

Let $\mathbf{A}$ be a $2^n \times 2^n$ matrix with a single non-zero entry in position (r, c) $r, c \in \{0, 1, \dots, 2^n - 1\}$. Let the binary representation of $r = \sum_{p=0}^{n-1} 2^p q_r(p)$ and $c = \sum_{p=0}^{n-1} 2^p q_c(p)$. Then, the tensor decomposition of $\mathbf{A}$ in sigma basis is given by $\mathbf{A}_s = \mathbf{A}_{rc} (\otimes_p \sigma(q_r(p), q_c(p)))$ where,

$$\sigma(q_r(p), q_c(p)) = \begin{cases} \sigma_+ \sigma_- & \text{if } q_r(p) = 0, q_c(p) = 0 \\ \sigma_+ & \text{if } q_r(p) = 0, q_c(p) = 1 \\ \sigma_- & \text{if } q_r(p) = 1, q_c(p) = 0 \\ \sigma_- \sigma_+ & \text{if } q_r(p) = 1, q_c(p) = 1 \end{cases}.$$

Sigma Basis Decomposition: Computational Methods

- **Numerical:** Express $\mathbf{A}$ as a sum of matrices $\tilde{\mathbf{A}}_i \in \mathbb{C}^{2^n \times 2^n}$ each with a single non-zero entry, i.e.

$$\mathbf{A} = \sum_{i=1}^{\text{nnz}(\mathbf{A})} \tilde{\mathbf{A}}_i,$$

and invoke previous theorem for each $\tilde{\mathbf{A}}_i$.

- With sigma basis decomposition the #terms is always upper-bounded by $\text{nnz}(\mathbf{A})$.
- $\mathbf{I} \in \mathbb{S}$ is not used and this approach may not produce a decomposition with the minimum number of terms.
- **Semi-Analytical:** Mix of analytical and numerical: analytical for recursive/ telescoping matrix structure and numerical for the stay entries

$$\mathbf{A}^{(2)} = \begin{bmatrix} \star & \star \\ \star & \star \end{bmatrix} \quad \mathbf{A}^{(4)} = \left[\begin{array}{c|c} \mathbf{A}^{(2)} & \star \\ \hline \star & \mathbf{A}^{(2)} \end{array} \right] \quad \mathbf{A}^{(8)} = \left[\begin{array}{c|c} \mathbf{A}^{(4)} & \star \\ \hline \star & \mathbf{A}^{(4)} \end{array} \right]$$

Unitary Completion Concept

- Given some elements of $\mathbb{S}$ are non-unitary, how to implement

$$\mathbf{A}_l = \sigma_{n-1} \otimes \sigma_{n-2} \otimes \cdots \otimes \sigma_0 \equiv \otimes_k \sigma_k$$

in form of a quantum circuit?

- There are many options, e.g., unitary dilation, unitary completion⁴ etc.

Definition

If $\mathbf{U} : W \rightarrow V$ is a linear operator which preserves inner products, i.e., for any $|w_1\rangle, |w_2\rangle$ in $W \subset V$, $\langle w_1 | \mathbf{U}^* \mathbf{U} | w_2 \rangle = \langle w_1 | w_2 \rangle$, then

- An unitary operator $\bar{\mathbf{U}} : V \rightarrow V$ is the unitary completion of $\mathbf{U}$, if $\bar{\mathbf{U}}$ spans the whole space V and $\bar{\mathbf{U}} |w\rangle = \mathbf{U} |w\rangle, \forall |w\rangle \in W$.
- $\mathbf{U}^\perp := \bar{\mathbf{U}} - \mathbf{U}$ is the orthogonal complement of $\mathbf{U}$.

Such a unitary operator $\bar{\mathbf{U}}$ always exists.

⁴A. Nielsen and I. L. Chuang, Quantum computation and quantum information, Cambridge University Press, 2010

Unitary Completion for Sigma Basis

- The unique unitary completion/complements for sigma basis element

Basis element	Unitary Completion	Orthogonal Complement
σ_+	σ_x	σ_-
σ_-	σ_x	σ_+
$\sigma_+\sigma_-$	I	$\sigma_-\sigma_+$
$\sigma_-\sigma_+$	I	$\sigma_+\sigma_-$
I	I	0

- Let $\mathbf{A}_l = \otimes_k \sigma_k$, define

$$\bar{\mathbf{A}}_l := \otimes_k \bar{\sigma}_k, \quad \bar{\sigma}_k \in \{\sigma_x, \mathbf{I}\} \text{ is unitary completion of } \sigma_k \quad (2)$$

$$\mathbf{A}_l^\perp := \bar{\mathbf{A}}_l - \mathbf{A}_l.$$

- Then it can be proved that
 - $\bar{\mathbf{A}}_l$ is unitary completion of $\mathbf{A}_l$.
 - $\mathbf{A}_l^\perp$ is orthogonal complement of $\mathbf{A}_l$.
 - $\mathbf{A}_l^T \mathbf{A}_l^\perp = \mathbf{0}$
 - $\mathbf{A}_l^T \mathbf{A}_l + (\mathbf{A}_l^\perp)^T \mathbf{A}_l^\perp = \mathbf{I}$

Unitary Completion for Sigma Basis

- Consider an operator $\mathbf{U}_l$ associated with operator $\mathbf{A}_l$ of the form

$$\mathbf{U}_l = \begin{bmatrix} \mathbf{A}_l & \mathbf{A}_l^\perp \\ \mathbf{A}_l^\perp & \mathbf{A}_l \end{bmatrix},$$

then, $\mathbf{U}_l$ is a unitary operator whose application on an ancilla system $|0\rangle |\psi\rangle$ leads to

$$\mathbf{U}_l |0\rangle |\psi\rangle = |0\rangle \mathbf{A}_l |\psi\rangle + |1\rangle \mathbf{A}_l^\perp |\psi\rangle.$$

- Example: $\mathbf{A}_l = \sigma_+ \sigma_-$

$$\mathbf{A}_l = \sigma_+ \sigma_- = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad \overline{\mathbf{A}}_l = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{A}_l^\perp = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{U}_l = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}.$$

Unitary Completion for Sigma Basis

- It is tedious to work with $\mathbf{A}_l^\perp$ (as it does not have closed form tensor product expression), so we equivalently express $\mathbf{U}_l$ as $\mathbf{U}_l = \mathbf{U}_{l,1}\mathbf{U}_{l,2}$ with

$$\mathbf{U}_{l,1} = \begin{bmatrix} \mathbf{I} - \mathbf{A}_l\mathbf{A}_l^T & \mathbf{A}_l\mathbf{A}_l^T \\ \mathbf{A}_l\mathbf{A}_l^T & \mathbf{I} - \mathbf{A}_l\mathbf{A}_l^T \end{bmatrix}, \quad \mathbf{U}_{l,2} = \sigma_x \otimes \bar{\mathbf{A}}_l.$$

- Note that $\mathbf{A}_l\mathbf{A}_l^T$ is a binary diagonal matrix with 1's and 0's at the diagonal, since

$$\mathbf{A}_l\mathbf{A}_l^T = \otimes_p \sigma_p \sigma_p^T, \quad \sigma_p \sigma_p^T \in \{\sigma_+\sigma_-, \sigma_-\sigma_+, \mathbf{I}\}.$$

Therefore, $\mathbf{U}_{l,1}$ is a permutation matrix of size $2^{n+1} \times 2^{n+1}$, which can be implemented using only Toffoli gates.

- $\mathbf{U}_{l,2}$ only involves tensor products of $\sigma_x, \mathbf{I}$, and thus can be implemented efficiently using the single qubit gates.

Code walkthrough: `Session_1/code/lcnu_heat_equation.ipynb`

1. Derive LCNU decomposition using heat equation as an example
2. Construct the circuit for LCNU term
3. Use the constructed circuit in Hadamard tests



Clone the repo

We will end with a discussion on other related techniques in literature such as unitary dilation and sparse-access oracles and generalization to a linear combination of “things” framework.

Sigma basis notation

$$\sigma_+ = |0\rangle\langle 1| = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad \sigma_- = |1\rangle\langle 0| = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad \sigma_+\sigma_- = |0\rangle\langle 0| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad \sigma_-\sigma_+ = |1\rangle\langle 1| = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$$

One term is $\mathbf{A}_l = \sigma_0 \otimes \cdots \otimes \sigma_{n-1}$, encoded as a string of n letters.

letter	operator	(row bit, column bit)
P	$\sigma_+\sigma_-$	(0, 0)
+	σ_+	(0, 1)
-	σ_-	(1, 0)
M	$\sigma_-\sigma_+$	(1, 1)
I	I	(0, 0) and (1, 1)

Heat Equation

A rod conducts heat. A constant flux q enters at $x = 0$. The end at $x = l$ is insulated.

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}, \quad -k \left. \frac{\partial u}{\partial x} \right|_{x=0} = q, \quad -k \left. \frac{\partial u}{\partial x} \right|_{x=l} = 0.$$

Central differences in space, backward Euler in time. One step becomes

$$(\mathbf{I} - c \mathbf{A}') \mathbf{u}_j = \mathbf{u}_{j-1} + \frac{q \Delta t}{k \Delta x} \mathbf{e}_1, \quad c = \frac{\alpha \Delta t}{(\Delta x)^2},$$

with $\mathbf{A}'$ the Laplacian carrying a zero-flux stencil.

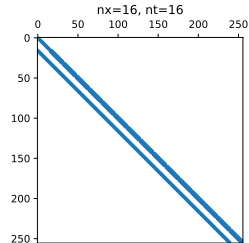
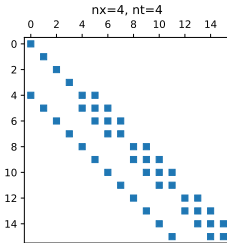
Extended linear system: Stack every time level into one vector, so a quantum linear solver solves a single $\mathbf{A} \mathbf{u} = \mathbf{b}$.

Linear System

$$\mathbf{A} = \underbrace{\begin{pmatrix} \mathbf{I} & & \\ -\mathbf{I} & \ddots & \\ & -\mathbf{I} & \mathbf{I} \end{pmatrix}}_{\mathbf{A}_1} - c \underbrace{\begin{pmatrix} \mathbf{0} & & \\ & \ddots & \\ & & \mathbf{A}' \end{pmatrix}}_{\mathbf{A}_2}$$

Block row 0 sets the initial condition. So the first block of $\mathbf{A}_2$ is **zero**. Hence, the first block of $\mathbf{A}$ has a different pattern.

Set $n_x = 2^s$ and $n_t = 2^t$. Then $\mathbf{A}$ acts on $n = s + t$ qubits, and the time index takes the high bits.



Hand-computed decomposition

Use a recursive decomposition strategy - half the matrix size at every step, hence $\log_2 N$ levels of decomposition.

$$\mathbf{A}_1 = \mathbf{I}^{\otimes n} - \sum_{j=1}^t \mathbf{I}^{\otimes(j-1)} \otimes \sigma_- \otimes \sigma_+^{\otimes(t-j)} \otimes \mathbf{I}^{\otimes s} \quad (\log_2 n_t + 1)$$

$$\mathbf{A}' = -\mathbf{T} + (\sigma_+ \sigma_-)^{\otimes s} + (\sigma_- \sigma_+)^{\otimes s} \quad (2 \log_2 n_x + 3)$$

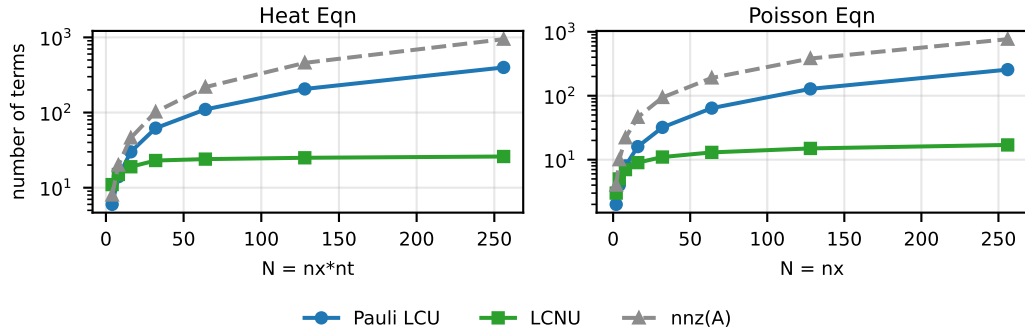
$$\mathbf{A}_2 = (\mathbf{I}^{\otimes t} - (\sigma_+ \sigma_-)^{\otimes t}) \otimes \mathbf{A}' \quad (4 \log_2 n_x + 6)$$

$\mathbf{T}$ is the Poisson matrix at $2 \log_2 n_x + 1$ terms. Each non-zero entry that do not fall into the repeating pattern, can be written as a single LCNU term.

Total: $\log_2 n_t + 4 \log_2 n_x + 7$ terms

The last $\mathbf{A}_2$ exists because the initial condition doesn't follow the repeating pattern of the rest of the matrix.

Number of LCU (Pauli) and LCNU terms



- Pauli wins below $N = 16$. The two cross near $N = 16$, then LCNU pulls away.
- At $n_x = n_t = 16$: **398 Pauli terms against 26.**
- For the Poisson matrix the Pauli count is exactly n_x , linear in the matrix size.
- The $\text{nnz}(\mathbf{A})$ curve is the Theorem 1 bound. It sits above Pauli everywhere.

Unitary completion

Four of the five basis elements are not unitary, so no circuit builds them directly.

σ	$\bar{\sigma}$	σ^c
σ_+	σ_x	σ_-
σ_-	σ_x	σ_+
$\sigma_+\sigma_-$	$\mathbf{I}$	$\sigma_-\sigma_+$
$\sigma_-\sigma_+$	$\mathbf{I}$	$\sigma_+\sigma_-$
$\mathbf{I}$	$\mathbf{I}$	$\mathbf{0}$

Set $\bar{\mathbf{A}}_l = \otimes_k \bar{\sigma}_k$ and $\mathbf{A}_l^c = \bar{\mathbf{A}}_l - \mathbf{A}_l$. Then

$$\mathbf{U}_l = \begin{pmatrix} \mathbf{A}_l & \mathbf{A}_l^c \\ \mathbf{A}_l^c & \mathbf{A}_l \end{pmatrix}$$

is unitary, and

$$\mathbf{U}_l|0\rangle|\psi\rangle = |0\rangle\mathbf{A}_l|\psi\rangle + |1\rangle\mathbf{A}_l^c|\psi\rangle.$$

$\mathbf{A}_l^c$ has no tensor product form, so factor $\mathbf{U}_l = \mathbf{U}_{l,1}\mathbf{U}_{l,2}$:

$$\mathbf{U}_{l,1} = \begin{pmatrix} \mathbf{I} - \mathbf{D} & \mathbf{D} \\ \mathbf{D} & \mathbf{I} - \mathbf{D} \end{pmatrix}, \quad \mathbf{D} = \mathbf{A}_l\mathbf{A}_l^T, \quad \mathbf{U}_{l,2} = \sigma_x \otimes \bar{\mathbf{A}}_l.$$

$\mathbf{D}$ is diagonal with 0s and 1s, so $\mathbf{U}_{l,1}$ is one multi-controlled X gate.

n -controlled Toffoli gate

We use C^nX to represent the n -controlled Toffoli gate:

- C^nX gate acts on $n + 1$ qubits and swaps two rows whose binary representation differ in the k -th bit,
 - with k -th bit being the target qubit, and
 - open (closed) control at $p \neq k$ -th qubit implies 0(1) bit at the p -th location in the binary representation of the row index.
- C^1X is the CNOT gate and C^2X is the Toffoli gate or CCNOT gate.

Circuit	Truth Table	Circuit	Truth Table																																																																																																												
	<table><tr><th>q_0</th><th>q_1</th><th>q_2</th><th>q'_0</th><th>q'_1</th><th>q'_2</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	q_0	q_1	q_2	q'_0	q'_1	q'_2	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	1	0	0	1	1	0	1	1	1	0	0	1	0	0	1	0	1	1	0	1	1	1	0	1	1	1	1	1	1	1	1	0		<table><tr><th>q_0</th><th>q_1</th><th>q_2</th><th>q'_0</th><th>q'_1</th><th>q'_2</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	q_0	q_1	q_2	q'_0	q'_1	q'_2	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	1	0	0	1	1	0	1	1	1	0	0	1	0	0	1	0	1	1	0	1	1	1	0	1	1	0	1	1	0	1	1	0
q_0	q_1	q_2	q'_0	q'_1	q'_2																																																																																																										
0	0	0	0	0	0																																																																																																										
0	0	1	0	0	1																																																																																																										
0	1	0	0	1	0																																																																																																										
0	1	1	0	1	1																																																																																																										
1	0	0	1	0	0																																																																																																										
1	0	1	1	0	1																																																																																																										
1	1	0	1	1	1																																																																																																										
1	1	1	1	1	0																																																																																																										
q_0	q_1	q_2	q'_0	q'_1	q'_2																																																																																																										
0	0	0	0	0	0																																																																																																										
0	0	1	0	0	1																																																																																																										
0	1	0	0	1	0																																																																																																										
0	1	1	0	1	1																																																																																																										
1	0	0	1	0	0																																																																																																										
1	0	1	1	0	1																																																																																																										
1	1	0	1	1	0																																																																																																										
1	1	0	1	1	0																																																																																																										
	<table><tr><th>q_0</th><th>q_1</th><th>q_2</th><th>q'_0</th><th>q'_1</th><th>q'_2</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td></tr></table>	q_0	q_1	q_2	q'_0	q'_1	q'_2	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	1	0	0	1	1	0	1	1	1	0	0	1	0	0	1	0	1	1	0	1	1	1	0	1	1	0	1	1	1	1	1	1		<table><tr><th>q_0</th><th>q_1</th><th>q_2</th><th>q'_0</th><th>q'_1</th><th>q'_2</th></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	q_0	q_1	q_2	q'_0	q'_1	q'_2	0	0	0	0	0	0	0	0	1	0	0	1	0	1	0	0	1	0	0	1	1	0	1	0	1	0	0	1	0	0	1	0	1	1	0	1	1	1	0	1	1	0	1	1	0	1	1	0
q_0	q_1	q_2	q'_0	q'_1	q'_2																																																																																																										
0	0	0	0	0	0																																																																																																										
0	0	1	0	0	1																																																																																																										
0	1	0	0	1	0																																																																																																										
0	1	1	0	1	1																																																																																																										
1	0	0	1	0	0																																																																																																										
1	0	1	1	0	1																																																																																																										
1	1	0	1	1	0																																																																																																										
1	1	1	1	1	1																																																																																																										
q_0	q_1	q_2	q'_0	q'_1	q'_2																																																																																																										
0	0	0	0	0	0																																																																																																										
0	0	1	0	0	1																																																																																																										
0	1	0	0	1	0																																																																																																										
0	1	1	0	1	0																																																																																																										
1	0	0	1	0	0																																																																																																										
1	0	1	1	0	1																																																																																																										
1	1	0	1	1	0																																																																																																										
1	1	0	1	1	0																																																																																																										

Circuit construction

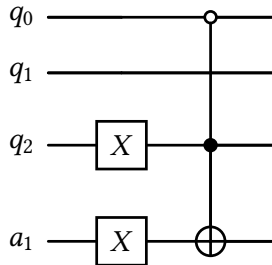
Algorithm 1: circuit for U_l

Require: $A_l = \sigma_{n-1} \otimes \sigma_{n-2} \otimes \cdots \otimes \sigma_0$

Require: $n + 1$ qubits $q_0, \dots, q_{n-1}$, ancilla a_0

1. Compute $\bar{A}_l = \otimes_k \bar{\sigma}_k$, where $\bar{\sigma}_k \in \{\sigma_x, \mathbf{I}\}$
2. Apply the n single-qubit gates of $\bar{A}_l$
3. Apply X on a_0
4. Set $k = \sum_{p=0}^{n-1} [\sigma_p = \mathbf{I}]$, with $[\cdot]$ equal to 1 when true and 0 when false
5. Construct a $C^{n-k}X$ gate:
 for $i \leftarrow 0$ **to** $n - 1$:
 if $\sigma_i \in \{\sigma_-, \sigma_- \sigma_+\}$: closed control on q_i
 if $\sigma_i \in \{\sigma_+, \sigma_+ \sigma_-\}$: open control on q_i
 end, then target X on a_0

Example. $A_l = \sigma_- \otimes \mathbf{I} \otimes \sigma_+ \sigma_-$, so $\bar{A}_l = \sigma_x \otimes \mathbf{I} \otimes \mathbf{I}$ and $k = 1$.



Position 0 is the most significant factor, so the $-$ acts on q_2 .
The P is position 2, so it acts on q_0 .

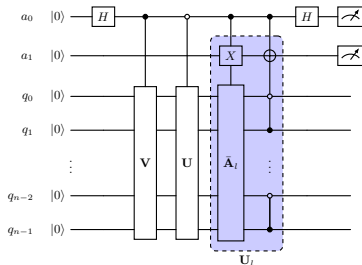
Qiskit puts the most significant qubit last, so a_1 is the bottom wire.

How to use with VQAs?

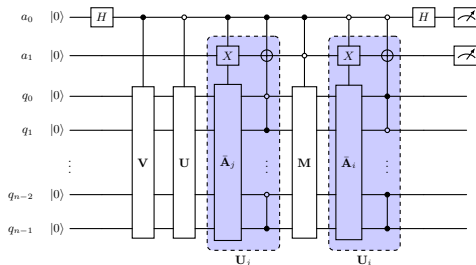
VQAs require various inner products of the following forms:

$$\langle \psi_1 | \mathbf{A}_l | \psi_2 \rangle, \quad \langle \psi_1 | \mathbf{A}_l^\dagger \mathbf{M} \mathbf{A}_j | \psi_2 \rangle, \quad |\psi_1\rangle = \mathbf{U}|0\rangle, \quad |\psi_2\rangle = \mathbf{V}|0\rangle.$$

Since $\mathbf{A}_l$ is not unitary, use $\mathbf{U}_l$, then discard the $a_l = 1$ branch.



(a) Circuit for computing $\langle 0 | \mathbf{U}^* \mathbf{A}_l \mathbf{V} | 0 \rangle$.



(b) Circuit for computing $\langle 0 | \mathbf{U}^* \mathbf{A}_l^* \mathbf{M} \mathbf{A}_j \mathbf{V} | 0 \rangle$.

Measure the ancillas to get the inner products

The state before the last H is

$$\frac{1}{\sqrt{2}}(|00\rangle|\psi_1\rangle + |10\rangle\mathbf{A}_l|\psi_2\rangle + |11\rangle\mathbf{A}_l^c|\psi_2\rangle).$$

After the H , the two outcomes with $a_1 = 0$ carry the answer:

$$P_{00} - P_{10} = \frac{1}{4} \left(\left\| |\psi_1\rangle + \mathbf{A}_l|\psi_2\rangle \right\|^2 - \left\| |\psi_1\rangle - \mathbf{A}_l|\psi_2\rangle \right\|^2 \right) = \text{Re} \langle \psi_1 | \mathbf{A}_l | \psi_2 \rangle.$$

In the notebook

Every test agrees with dense linear algebra to 10^{-16} .

Assembling the VQLS cost function

Take $|b\rangle = \mathbf{U}|0\rangle$, $|\psi\rangle = \mathbf{V}(\theta)|0\rangle$ and $|\phi\rangle = \mathbf{A}|\psi\rangle$.

$$C_g = 1 - \frac{|\langle b|\phi\rangle|^2}{\langle\phi|\phi\rangle}, \quad \langle\phi|\phi\rangle = \sum_{i,j} \alpha_i \bar{\alpha}_j \beta_{ij}, \quad |\langle b|\phi\rangle|^2 = \sum_{i,j} \alpha_i \bar{\alpha}_j \gamma_{ij}.$$

- $\gamma_{ij} = c_i \bar{c}_j$ with $c_i = \langle b|\mathbf{A}_i|\psi\rangle$. It factorizes, so it costs only n_l single-operator tests.
- $\beta_{ij} = \langle\psi|\mathbf{A}_j^\dagger \mathbf{A}_i|\psi\rangle$ does not factorize. It is Hermitian, so $n_l(n_l + 1)/2$ pair tests suffice.

The local cost function resists barren plateaus, and it adds a factor of n :

$$C_l = 1 - \frac{1}{n\langle\phi|\phi\rangle} \sum_{k=1}^n \sum_{i,j} \alpha_i \bar{\alpha}_j \frac{\beta_{ij} + \delta_{ijk}}{2}, \quad \mathbf{M} = \mathbf{U}(Z_k \otimes \mathbf{I})\mathbf{U}^\dagger.$$

In the afternoon session, we will show a demo with the complete VQLS setup for solving Burgers' equation.

Use in fault-tolerant algorithms

Block encoding is the data access model for fault-tolerant algorithms, and LCU is the usual way to build one. LCNU drops into the same PREP and SELECT routines through $\mathbf{U}_l$:

$$\text{PREP } |0\rangle = \sum_l \sqrt{\alpha_l} |l\rangle, \quad \text{SELECT } |l\rangle|0\rangle|\psi\rangle = |l\rangle \mathbf{U}_l |0\rangle|\psi\rangle.$$

PREP is untouched. SELECT gains one ancilla, the one that block-encodes $\mathbf{A}_l$ inside $\mathbf{U}_l$.

protocol	n_{anc}	count	depth
PREP	$\Omega(\log L) \leq n_{\text{anc}} \leq O(L)$	$O(L \log 1/\epsilon)$	$\tilde{O}(L \log(1/\epsilon) \log n_{\text{anc}}/n_{\text{anc}})$
SELECT	$\Omega(\log L + \log N) \leq n_{\text{anc}} \leq O(L \log N)$	$O(L \log N)$	$O(L \log N \log n_{\text{anc}}/n_{\text{anc}})$

Clifford+T complexity, for $|\alpha\rangle$ of size L . Here ϵ is the accuracy of the state preparation, and $\tilde{O}$ hides doubly logarithmic factors.

The Sigma basis reaches **the same asymptotics**, and pays two extra ancilla bits. So the cost turns on L alone. With $L = O(N)$ for Pauli and $L = O(\text{poly log } N)$ for Sigma, the gain in count, depth and ancillas is exponential.

This afternoon. Numerical T-count estimates for block-encoding the linear system from Burgers' equation.

Algorithmic decomposition into LCNU

Everything so far came from a hand-compute recursion. T

Given a sparse matrix, algorithmically find a short LCNU decomposition.

Run the same block split as an algorithm. Halve on the most significant qubit:

$$\mathbf{B} = \begin{pmatrix} \mathbf{B}_{00} & \mathbf{B}_{01} \\ \mathbf{B}_{10} & \mathbf{B}_{11} \end{pmatrix} = \sigma_+ \sigma_- \otimes \mathbf{B}_{00} + \sigma_- \sigma_+ \otimes \mathbf{B}_{11} + \sigma_+ \otimes \mathbf{B}_{01} + \sigma_- \otimes \mathbf{B}_{10}$$

A zero block drops out. Each level halves the matrix, so the depth is $\log_2 N$.

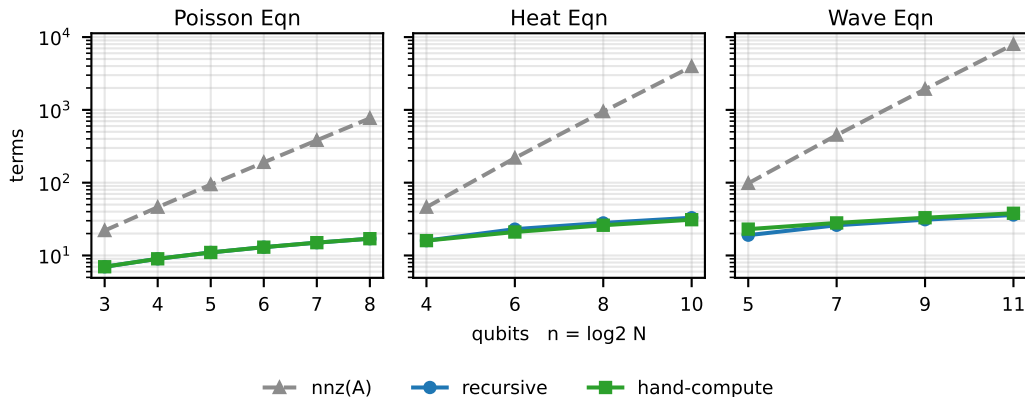
Algorithmic recursion:

Identify the common block $\mathbf{C}$ between $\mathbf{B}_{00}, \mathbf{B}_{11}$

$$\sigma_+ \sigma_- \otimes \mathbf{B}_{00} + \sigma_- \sigma_+ \otimes \mathbf{B}_{11} = \mathbf{I} \otimes \mathbf{C} + \sigma_+ \sigma_- \otimes (\mathbf{B}_{00} - \mathbf{C}) + \sigma_- \sigma_+ \otimes (\mathbf{B}_{11} - \mathbf{C})$$

$\mathbf{C}$ can be chosen as either $\mathbf{B}_{00}$ or $\mathbf{B}_{11}$ or entry-wise the smallest magnitude of $\mathbf{B}_{00}, \mathbf{B}_{11}$. These are some heuristics based on hand-crafting the recursion.

How well it does



- Exact everywhere, to machine precision.
- Close to hand-crafted decomposition

$n_x = n_t = 32$	recur.	hand	nnz
heat	33	31	3938
wave	36	38	7938

Comparison with Unitary Dilation and Sparse Access Oracles

Sparse-access oracles. Every contraction has a dilation. For $\|\mathbf{A}\|_2 \leq 1$ with $D_{\mathbf{A}} = \sqrt{\mathbf{I} - \mathbf{A}^* \mathbf{A}}$, and for a Sigma basis term, where $(\mathbf{I} - \mathbf{A}_l^* \mathbf{A}_l)^2 = \mathbf{I} - \mathbf{A}_l^T \mathbf{A}_l$,

$$\tilde{\mathbf{U}}_{\mathbf{A}} = \begin{pmatrix} \mathbf{A} & D_{\mathbf{A}^*} \\ D_{\mathbf{A}} & -\mathbf{A}^* \end{pmatrix} \longrightarrow \tilde{\mathbf{U}}_l = \begin{pmatrix} \mathbf{A}_l & \mathbf{I} - \mathbf{A}_l \mathbf{A}_l^T \\ \mathbf{I} - \mathbf{A}_l^T \mathbf{A}_l & -\mathbf{A}_l^T \end{pmatrix}$$

Both $\mathbf{U}_l$ and $\tilde{\mathbf{U}}_l$ are permutation matrices, so both are built from multi-controlled X gates. However, completion is more efficient.

	single-qubit gates	$C^m X$ gates, $m \leq n$
unitary completion, $\mathbf{U}_l$	at most $n + 1$	1
unitary dilation, $\tilde{\mathbf{U}}_l$	1	$2s + 1$

Here $s = \sum_i [\sigma_i \in \{\sigma_+, \sigma_-\}]$ counts the off-diagonal letters in the term.

Sparse-access oracles. These require *expert analysis and hand-crafted oracle construction*, tailored to the specific sparsity and structure of the problem. Our framework adapts to the structure of the matrix without the need for oracle engineering.

Comparison with related works

We are not the first or the only ones to come up with this basis. The idea originates from Bell basis.

- **Liu et al.** (2021), PRA **104**, 022418, arXiv:2012.07014
 - Construct a Hermitian observable per basis element by a Bell measurement.
 - Each observable is built for one term of one cost function. A different inner product needs another observable, and it is not obvious how to construct one.
- **Hu, Jin, Liu, Zhang** (2024), *Quantum* **8**, 1563, arXiv:2403.10032,
Sato et al. (2024), PRResearch **6**, 033246, arXiv:2402.18398
 - Heat equation combined with Schrödingerisation technique; Wave and advection equations
 - Trotter splitting of the shift operators, then explicit gates.
 - Non-unitary terms have to be paired and each Hermitian pair is diagonalised in a Bell basis. This restricts the PDEs that the decomposition can serve.

Our LCNU framework provides,

- **Generalizability:** Unitary completion makes each non-unitary term a unitary, so the result is an LCU-style primitive. Hadamard tests, block encoding and QSVT use it unchanged.
- No restriction on combinability of terms, no need to hand-craft observables to use downstream.

Linear Combination of “Things”

The basis is a choice, not a constraint. Do not limit to thinking in one basis or another.

You can construct **non-unitary** basis elements can be built to order, from the row spaces of a unitary. Take the Hadamard matrix:

$$\sigma_{H,1} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, \quad \sigma_{H,2} = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 0 \\ 1 & -1 \end{bmatrix}, \quad \sigma_{H,1} + \sigma_{H,2} = H.$$

Both have unitary completion H , and $\sigma_{H,1}\sigma_{H,1}^T = \sigma_+\sigma_-$, $\sigma_{H,2}\sigma_{H,2}^T = \sigma_-\sigma_+$. The circuit construction therefore applies with no change at all.

The idea goes beyond 2×2 unitaries and non-unitaries. **Come back in the afternoon** to learn about,

- Extension of the framework incorporating a bigger basis set and general theory
- Hands-on session in solving the Carleman linearized Burgers' equation (non-trivial sparsity pattern!)

1. A. Gnanasekaran and A. Surana, *Efficient variational quantum linear solver for structured sparse matrices*, IEEE International Conference on Quantum Computing and Engineering (QCE), vol. 1, pp. 199–210, 2024, [arXiv:2404.16991](#).
2. A. Gnanasekaran and A. Surana, *Efficient quantum access model for sparse structured matrices using linear combination of “things”*, Physical Review A **113**, 022437, 2026, [arXiv:2507.03714](#).
3. R. Demirdjian, T. Hogancamp and D. Gunlycke, *Efficient decomposition of the Carleman linearized Burgers’ equation*, Physical Review A **113**, 032408, 2026, [arXiv:2505.00285](#).
4. R. Demirdjian, T. Hogancamp, A. Gnanasekaran, A. Surana and D. Gunlycke, *Quantum Data Loading for Carleman Linearized Systems: Application to the Lattice-Boltzmann Equation*, 2026 [arXiv:2605.00302](#).